

Work Summary: Qwen Multi-Agent Client & Server

1. Project Overview

This project establishes a robust, local LLM infrastructure using the Qwen model series. It consists of a FastAPI-based server handling model inference and a Python-based CLI client for user interaction. The system is designed to leverage high-performance hardware (specifically optimizing for VRAM usage on consumer/prosumer GPUs) to run multiple specialized AI agents.

2. Key Features

- **Multi-Agent Architecture:** The system supports distinct agents with specialized roles:
 - **Implementer:** Optimized for code generation.
 - **Architect:** Uses a larger model (480B) for high-level system design.
 - **Reviewer:** Specialized for code analysis.
 - **Debugger:** Focused on error resolution.
- **Remote Command Execution:** The client allows agents to execute shell commands on the user's machine (with safety protocols like allow-lists and confirmation prompts).
- **Context Management:** Features long-term memory via vector storage and “smart reloading” to balance VRAM usage with response latency.
- **Cross-Platform Client:** The CLI client (`qwen_remote.py`) works on both Linux and macOS, handling environment differences automatically.

3. Architecture & Technical Configuration

- **Server (`server.py`):**
 - Built with FastAPI for OpenAI-compatible endpoints.
 - Manages model loading/unloading to prevent Out-Of-Memory (OOM) errors, swapping models dynamically based on the requested agent.
 - Supports advanced `llama.cpp` parameters like YaRN scaling for extended context windows (up to 512k tokens).
 - Implements `StartLimitIntervalSec` and `StartLimitBurst` in `systemd` to prevent restart loops.
- **Client (`qwen_remote.py`):**
 - Interactive CLI with `readline` support for command history.
 - Supports async command execution for long-running tasks.
 - Includes “Smart Reloading” logic to keep models loaded during interactive sessions but reload them when switching contexts.

4. Development Timeline (Jan 2026)

Phase 1: Foundation & Optimization (Jan 3 - Jan 5)

- **Initial Setup:** Established the FastAPI server and basic client connectivity.
- **VRAM Optimization:** Significant effort was put into tuning model parameters (`n_gpu_layers`, `n_batch`) to fit models within available VRAM.
- **Agent Specialization:** Introduced the concept of distinct models for different roles (e.g., using a smaller, faster model for implementation and a massive 480B model for architecture).
- **Context Expansion:** Increased context windows (up to 512k) and adjusted batch sizes to trade off speed for capacity.

Phase 2: Client Enhancements & Stability (Jan 24 - Jan 27)

- **History & Usability:** Added persistent chat history (`~/.qwen_chat_history.json`) and command history support.
- **Performance Tuning:** Refined server thread counts and added VRAM critical settings.
- **Bug Fixes:** Resolved issues with command history artifacts on macOS/Linux clients and fixed model paths (specifically correcting the `architect` model path).
- **Service Hardening:** Updated `qwen-server.service` with better timeout and restart policies to handle heavy model unloading gracefully.
- **Feature Additions:**
 - Added `@agent` syntax for quick agent switching.
 - Implemented persistent agent state (client remembers the last used agent).
 - Fixed readline artifacts for a cleaner UI.

5. Current Status

The project is functional and stable. The server safely manages resource-intensive models, and the client provides a smooth developer experience with features like history navigation, quick agent switching, and safe remote execution.